Birla Institute of Technology and Science, Pilani

Hyderabad Campus

Design and Analysis of Algorithms

CS F364

Assignment 1

Brandes' Algorithm for Betweenness Centrality

Implementation, Evaluation, and Large-Graph Runtime Analysis

S.No.	Name	Institute ID
1	Mayan Vajani	2022B3A70611H
2	Piyush Nahata	2022B3A70654H
3	Kanishk Agarwal	2022B3A71390H
4	Srinath Murali	2022B3A70594H
5	Yuvan Kanthamaneni	2022B3A70785H

Team Number: 4

Instructor: Dr. Apurba

Date of Submission: 20 March 2026

Contents

1 Assignment Requirements and Compliance **2**
1.1 Assignment brief (as given) . . . 2
1.2 Important instruction . . . 2
1.3 Deliverable coverage checklist . . . 2

2 Introduction **2**

3 Algorithm Summary **3**
3.1 Reference . . . 3
3.2 Core idea . . . 3
3.3 Two-phase dependency accumulation . . . 3
3.4 What is implemented from the paper . . . 3
3.5 Approximation for very large graphs . . . 4

4 Dataset Descriptions **4**
4.1 wiki-Vote . . . 4
4.2 email-Enron . . . 4
4.3 as-Skitter . . . 4

5 Implementation Details **4**
5.1 Language and execution . . . 4
5.2 Key implementation features . . . 4
5.3 Correctness verification . . . 5

6 Dataset Outputs **5**
6.1 Dataset 1: wiki-Vote . . . 5
6.2 Dataset 2: email-Enron . . . 5
6.3 Dataset 3: as-Skitter (final high-quality approximation) . . . 6

7 Execution Time and Space Summary **7**

8 Observations and Discussion **8**

9 Individual Contributions **8**

10 Reproducibility **8**
10.1 Compile . . . 8
10.2 Run all datasets . . . 8

11 Conclusion **9**

12 References **9**

1. Assignment Requirements and Compliance

1.1. Assignment brief (as given)

1. Implement Brandes' betweenness centrality algorithm in C/C++.
2. Execute on the provided SNAP datasets: `email-Enron`, `as-Skitter`, and `wiki-Vote`.
3. Report:
   - Top 20 nodes with betweenness centrality for each dataset.
   - Execution time on all three datasets.
   - Space consumption of the algorithm on all three datasets.

1.2. Important instruction

All graphs are treated as **unweighted and undirected**, as specified in the assignment note.

1.3. Deliverable coverage checklist

Required item	Status	Evidence in submission
Source code in C/C++	Done	`brandes.cpp`
Execution script for datasets	Done	`run.sh`
Team information (name + ID)	Done	Cover page table
Individual contribution statement	Done	Section 7
Top-20 output for all datasets	Done	Section 5 tables
Execution time for all datasets	Done	Section 6 summary table
Space consumption for all datasets	Done	Section 6 (measured for two datasets, estimated for as-Skitter)

2. Introduction

Betweenness centrality is a widely used measure in network analysis because it quantifies how frequently a vertex lies on shortest paths between other vertex pairs. Vertices with high betweenness often act as structural bridges that control information flow between communities.

Computing exact betweenness at scale is expensive. Naive all-pairs strategies become impractical on large graphs, while Brandes' algorithm provides a major improvement by combining shortest-path counting with dependency accumulation in a single-source framework. For unweighted graphs, this yields $O(nm)$ time and $O(n + m)$ space.

In this work, we implement and evaluate Brandes-based computation on three SNAP datasets under the assignment constraint that all graphs are treated as undirected and unweighted. Exact runs are reported for wiki-Vote and email-Enron; sampled-source approximation is used for as-Skitter due day-scale exact runtime on available hardware.

3. Algorithm Summary

3.1. Reference

Ulrik Brandes, *A faster algorithm for betweenness centrality*, Journal of Mathematical Sociology, 25(2):163–177, 2001.

3.2. Core idea

For each source vertex s, the algorithm performs:

1. A BFS traversal (unweighted case) to compute shortest-path counts $\sigma_s(\cdot)$, distances, and predecessor lists.
2. A reverse dependency accumulation phase to update centrality values.

For unweighted graphs, complexity is:

$$T = O(nm), \qquad S = O(n + m)$$

where n is number of vertices and m is number of edges.

3.3. Two-phase dependency accumulation

For each source s, the computation has two phases:

1. **Forward BFS phase**: compute distance $d(\cdot)$, shortest-path counts $\sigma(\cdot)$, and predecessor lists $P(\cdot)$.
2. **Reverse accumulation phase**: process vertices in reverse distance order and propagate dependencies using:

$$\delta_s(v) \mathrel{+}= \sum_{w:v\in P_s(w)} \frac{\sigma_s(v)}{\sigma_s(w)} \left(1 + \delta_s(w)\right)$$

The final betweenness score is obtained by summing dependencies over all sources; for undirected graphs, values are divided by 2 to avoid double counting.

3.4. What is implemented from the paper

The implementation follows Brandes' exact dependency-accumulation method for betweenness centrality:

1. Single-source shortest paths from each source vertex using BFS (unweighted case).
2. Computation of shortest-path counts σ and predecessor lists P during traversal.
3. Reverse accumulation of dependencies δ to update centrality values.
4. Final division by 2 for undirected graphs (as each pair is counted twice).

3.5. Approximation for very large graphs

For as-Skitter, we additionally use sampled-source approximation (Bader et al., 2007): instead of running all n sources, we run k randomly sampled sources and scale scores by n/k. This changes practical runtime from $O(nm)$ to $O(km)$ while preserving high-quality top-k rankings in practice.

4. Dataset Descriptions

All datasets are from the SNAP collection and are treated as undirected and unweighted.

4.1. wiki-Vote

Wikipedia voting network where nodes represent users and edges represent voting interactions. After preprocessing, the graph has 7,115 vertices and 100,762 undirected edges.

4.2. email-Enron

Email communication network inside Enron. Nodes are email addresses and edges represent communication links. After deduplication and undirected conversion, the graph has 36,692 vertices and 183,831 edges.

4.3. as-Skitter

Internet topology graph collected by CAIDA skitter. Nodes represent autonomous systems and edges represent observed connectivity. This is the largest graph with 1,696,415 vertices and 11,095,298 edges, and is the only dataset evaluated with sampled-source approximation.

5. Implementation Details

5.1. Language and execution

- Language: C++17
- Compiler: `g++ -O2`
- Platform used for measurements: Windows

5.2. Key implementation features

- Supports arbitrary input node IDs using internal ID remapping.
- Uses adjacency-list representation for linear-space graph storage.
- Handles SNAP edge-list comments and parsing robustly.
- Uses undirected treatment for assignment runs (default mode).
- Reports execution time and memory usage (estimated + measured peak RSS).

5.3. Correctness verification

The program includes built-in verification (`-verify`) against a naive baseline on small graphs, including path, star, complete, random, weighted, and directed random tests.

6. Dataset Outputs

6.1. Dataset 1: wiki-Vote

- Type used: Undirected, Unweighted
- Method: Exact Brandes (all sources; no sampling)
- Vertices: 7,115
- Edges: 100,762
- Execution time: 6.369 s
- Peak RSS: 14.6 MB

Rank	Node ID	Betweenness
1	2565	1549872.915482
2	11	902999.139004
3	457	897805.371404
4	4037	722673.553749
5	1549	661200.761785
6	766	641436.055712
7	1166	619015.072790
8	15	507140.272659
9	1374	483598.459253
10	2237	381003.275400
11	1151	358283.597568
12	2688	340525.449773
13	2328	309006.730450
14	2470	300141.675235
15	737	281894.381835
16	1186	254569.505328
17	5524	252635.392843
18	72	245368.826508
19	3352	244722.640597
20	2625	235889.026702

6.2. Dataset 2: email-Enron

- Type used: Undirected, Unweighted
- Method: Exact Brandes (all sources; no sampling)
- Vertices: 36,692
- Edges: 183,831

- Execution time: 82.594 s
- Peak RSS: 25.9 MB

Rank	Node ID	Betweenness
1	5038	43651092.833072
2	140	40660588.621809
3	566	24451044.780704
4	588	23867773.705425
5	1139	23832387.018878
6	273	23542249.801771
7	458	23227868.421221
8	46	20618684.654685
9	1028	19079252.525766
10	292	17177450.237447
11	195	17121514.623273
12	370	16954867.157902
13	823	14247752.804319
14	136	13926158.060870
15	76	13506564.342765
16	286	12731890.001473
17	734	12541114.657762
18	543	12539952.875823
19	1768	12533541.059658
20	353	11623777.314339

6.3. Dataset 3: as-Skitter (final high-quality approximation)

- Type used: Undirected, Unweighted
- Method: Approximate sampled-source Brandes (sampling used only for as-Skitter)
- Vertices: 1,696,415
- Edges: 11,095,298
- Approximation runtime (final run): 36,643.042 s (10.18 h)
- Samples used: 100,000 (seed 42)
- Estimated algorithm space: 246.43 MB

Reference exact command (impractical runtime on this machine):

```
.
brandes.exe -top 20 -o results_as-skitter.txt as-skitter.txt
```

On this machine, exact completion remains impractical in the submission window (probe-based estimate: 20.8–20.9 days), so the Top-20 table below reports the sampled-source run used for large-graph analysis.

Top-20 estimated betweenness for as-Skitter (approximate):

Rank	Node ID	Estimated Betweenness
1	5606	42340177305.4251
2	6736	39929141301.5368
3	6601	33020012013.1879
4	7121	25700236954.6217
5	5529	24285538261.3767
6	8881	21591705425.7688
7	628	21175456650.6645
8	6600	20901541310.5965
9	737	20266656035.7015
10	5624	19451782725.1703
11	1121	18207010399.3530
12	875	18072122874.6346
13	676	17720085826.6277
14	1039	17271390410.1580
15	674	16732430985.9725
16	24340	16265744117.3973
17	928	15381559604.6245
18	35	14830932213.6635
19	689	14605772454.6665
20	881	14139531171.2093

Calibration note: sampling on wiki-Vote and email-Enron was used only to tune approximation parameters; the reported Top-20 tables for those two datasets are exact Brandes outputs.

Convergence observations on as-Skitter (independent sampled runs):

Samples (k)	Runtime (s)	Top-1 Node	Top-20 overlap vs $k = 100000$
1000	290	5606	17/20
5000	1431	5606	19/20
12000	6935	5606	20/20
15000	4134	5606	20/20
100000	36643	5606	20/20 (reference)

7. Execution Time and Space Summary

Dataset	Vertices	Edges	Execution Time	Peak RSS
wiki-Vote	7,115	100,762	Exact: 6.369 s	14.6 MB
email-Enron	36,692	183,831	Exact: 82.594 s (1.4 min)	25.9 MB
as-Skitter	1,696,415	11,095,298	Approx (100,000): 36,643.042 s (10.18 h); exact estimated 20.8–20.9 days	246.43 MB (estimated)

Method split: wiki-Vote and email-Enron use exact Brandes values; only as-Skitter uses sampled approximation due exact day-scale runtime.

Estimated algorithm-space usage for as-Skitter from the final approximate run is 246.43 MB.

8. Observations and Discussion

1. **Scalability behavior:** Exact Brandes is practical for small-to-medium graphs (wiki-Vote, email-Enron), while sampled-source mode is necessary for as-Skitter under realistic deadlines.
2. **Space efficiency:** Observed memory usage remains linear in graph size ($O(n+m)$), which is the key advantage over naive all-pairs formulations.
3. **Ranking stability under sampling:** Across sampled runs, top-ranked as-Skitter nodes remain stable, supporting approximation quality for top-20 analysis.
4. **Network structure insight:** In all datasets, betweenness is highly skewed: a small set of vertices dominates shortest-path mediation.

9. Individual Contributions

Work was split into five primary tracks with comparable effort; each member owned one track and also participated in cross-review/testing.

Member	Contribution
Mayan Vajani	Implemented the core exact Brandes pipeline (BFS shortest paths, predecessor tracking, dependency accumulation), and integrated robust SNAP parsing under undirected/unweighted constraints.
Piyush Nahata	Built correctness workflow using baseline comparisons and `-verify` tests, and validated exact Top-20 outputs for wiki-Vote and email-Enron.
Kanishk Agarwal	Ran performance experiments, compiled time/space statistics across datasets, and tuned sampled-source settings for as-Skitter under submission-time constraints.
Srinath Murali	Structured the report narrative, converted raw outputs into publication tables, and ensured consistency between methods, claims, and reported metrics.
Yuvan Kanthamaneni	Prepared reproducible execution scripts and dataset setup flow, and synchronized final deliverables/artifacts across submission packages.

10. Reproducibility

10.1. Compile

```
g++ -O2 -std=c++17 -o brandes.exe brandes.cpp -lpsapi
```

10.2. Run all datasets

```
./run.sh
```

Script behavior: `run.sh` computes exact outputs for wiki-Vote, email-Enron, and as-Skitter using `brandes.cpp`. *Output files generated by run script:* `results_wiki-Vote.txt`, `results_email-Enron.txt`, and `results_as-skitter.txt`.
Optional exact as-Skitter command (very long runtime):

```
.
brandes.exe -top 20 -o results_as-skitter.txt
as-skitter.txt
```

11. Conclusion

The implementation satisfies the assignment constraint of treating all graphs as unweighted and undirected, and produces exact Top-20 centrality outputs for wiki-Vote and email-Enron along with measured time/space statistics. For as-Skitter, exact execution remains empirically estimated at about 20.8–20.9 days on this machine, so we provide a completed sampled Brandes run with $k = 100000$ (10.18 hours) and its Top-20 estimated centralities as the final practical large-graph result.

12. References

1. U. Brandes, "A faster algorithm for betweenness centrality," *Journal of Mathematical Sociology*, 25(2):163–177, 2001.

2. D. A. Bader, S. Kintali, K. Madduri, and M. Mihail, "Approximating Betweenness Centrality," in *WAW 2007*, LNCS 4863, pp. 124–137, Springer, 2007.

3. L. C. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, 40, 35–41, 1977.

4. L. C. Freeman, "Centrality in social networks: Conceptual clarification," *Social Networks*, 1, 215–239, 1979.

5. SNAP: Stanford Large Network Dataset Collection, https://snap.stanford.edu/data/.